BTS CIEL · PROGRAMMATION C++ · EXERCICES

Exercices — Chapitre 5 : Fonctions, séparation du code et tests

Cours associé : `C05_Fonctions_separation_tests` · **Fichiers de départ :** `02_Exercices/fichiers/E05/`

> **Méthode :** à partir de ce chapitre, une fonction de **traitement** n'affiche rien et ne lit rien au clavier : elle reçoit des paramètres et renvoie un résultat. C'est ce qui permet de la tester.

N°	Titre	Niveau	Notions du cours
5.1	Doubler une valeur	1 — Application	§5
5.2	Conversion de température	1 — Application	§1, §2
5.3	Interpréter des résultats de tests	2 — Entraînement	§9
5.4	Calculatrice en module	2 — Entraînement	§8
5.5	Contrôle de plage et tests	3 — Problème	§9
5.6	Module de mesures	4 — Défi	§6 à §9

Niveau 1 — Application directe

Exercice 5.1 — Doubler une valeur

Objectif : modifier une variable de l'appelant grâce à une référence.

Énoncé :

1. Écrivez `void doubler(int& valeur)`, qui double la valeur reçue.
2. Dans `main()`, affichez une variable avant et après l'appel pour vérifier que l'original est modifié.
3. Retirez le `&` du paramètre, recompilez et exécutez : qu'observez-vous ? Pourquoi ?

Résultat attendu (question 2) :

```
Avant : 21
Apres : 42
```

Exercice 5.2 — Conversion de température

Objectif : écrire une fonction avec paramètre et valeur de retour.

Énoncé :

1. Écrivez une fonction qui convertit des degrés Celsius en degrés Fahrenheit (F = 9/5 × C + 32). Son paramètre et sa valeur de retour sont de type `double`.
2. Appelez-la dans `main()` pour les valeurs du tableau.

3. Un collègue a écrit `9 / 5 * c + 32`. Pourquoi son résultat est-il faux ? Quelle valeur obtient-il pour 100 °C ?

Tests :

Celsius	Fahrenheit attendu
`0`	`32`
`100`	`212`
`-40`	`-40`
`37`	`98.6`

Niveau 2 — Entraînement

Exercice 5.3 — Interpréter des résultats de tests

Objectif : lire un tableau de tests et attribuer un écart à la fonction ou au test.

Énoncé : on teste `int convertirMinutesEnSecondes(int minutes)` :

Test	Entrée	Attendu	Obtenu	Verdict
1	2	120	120	?
2	0	0	0	?
3	5	360	300	?

1. Complétez la colonne « Verdict ».
2. Pour le test en échec, l'erreur vient-elle de la fonction ou du test ? Justifiez par un calcul.
3. Proposez un cas de test supplémentaire utile, et dites ce qu'il vérifie.

Exercice 5.4 — Calculatrice en module

Objectif : répartir un programme en un module (`.h` + `.cpp`) et un programme principal ; identifier une erreur d'édition de liens.

Énoncé :

1. Écrivez `additionner`, `soustraire` et `multiplier` : deux paramètres `int`, retour `int`.
2. Placez leurs prototypes dans `calculs.h`, avec une garde d'inclusion, et leurs définitions dans `calculs.cpp`.
3. Dans `main.cpp`, demandez deux entiers et affichez les trois résultats. Compilez les deux fichiers source.
4. Retirez `calculs.cpp` de la commande de compilation et relevez le message obtenu. À quelle étape apparaît-il ?

Contraintes : aucune fonction n'est définie dans `calculs.h` ; `calculs.cpp` inclut `calculs.h`.

Résultat attendu :

```
Premier entier : 7
Second entier : 3
7 + 3 = 10
7 - 3 = 4
7 * 3 = 21
```

Niveau 3 — Résolution de problèmes

Exercice 5.5 — Contrôle de plage et tests

Objectif : écrire une fonction de contrôle et la tester sur ses valeurs limites.

Énoncé :

1. Écrivez `bool tensionCorrecte(double tension)`, vraie si la tension est comprise entre 11,0 et 14,0 V incluses.
2. Écrivez une fonction de test sur le modèle du cours (§9) : elle affiche `OK` ou `ECHEC` pour chaque cas, puis le nombre d'échecs.
3. Prévoyez au moins six cas, dont les valeurs limites.
4. Introduisez volontairement une erreur (`<` au lieu de `<=`) : quel test la détecte ? Corrigez ensuite.

Contraintes : bornes en constantes `constexpr` ; `tensionCorrecte` n'affiche rien.

Résultat attendu : tous les tests réussis, et la réponse à la question 4.

Niveau 4 — Défi

Exercice 5.6 — Module de mesures

Objectif : réaliser un module complet à partir d'une spécification, avec `struct`, références constantes et tests.

Contexte : l'équipe logicielle d'un système de supervision a rédigé l'en-tête du module de mesures. Votre travail : écrire son implémentation et ses tests. Le fichier `mesures.h` vous est fourni :

```cpp
// Fichier : mesures.h
#ifndef MESURES_H
#define MESURES_H

#include <string>
#include <vector>

struct Mesure {
    std::string capteur{};
    double valeur{};
};

// Moyenne des valeurs ; 0.0 si le vecteur est vide
double moyenne(const std::vector<Mesure>& mesures);

// Copie dans resultat la mesure de plus grande valeur ; false si le vecteur est vide
bool chercherMaximum(const std::vector<Mesure>& mesures, Mesure& resultat);

// Nombre de mesures hors de la plage [min ; max]
int compterHorsPlage(const std::vector<Mesure>& mesures, double min, double max);

#endif
```

Énoncé :

1. Écrivez `mesures.cpp`, qui définit les trois fonctions.
2. Écrivez `test_mesures.cpp`, dont `main()` teste les trois fonctions sur les jeux de données ci-dessous et affiche le nombre d'échecs.
3. Compilez avec `g++ -std=c++17 -Wall -Wextra test_mesures.cpp mesures.cpp -o test_mesures`.

Contraintes :

- ne modifiez pas `mesures.h` ;
- aucune fonction du module n'affiche ni ne lit quoi que ce soit ;
- les résultats de type `double` sont comparés avec une tolérance (`std::fabs`, en-tête `<cmath>`).

Tests :

Jeu A : `{"T1", 21.5}`, `{"T2", 22.0}`, `{"T3", 85.3}`, `{"H1", -41.0}` ; plage de -40 à 85.

Fonction	Résultat attendu
`moyenne`	`21.95`
`chercherMaximum`	`true`, mesure `T3` de valeur `85.3`
`compterHorsPlage`	`2`

Jeu B : vecteur vide.

Fonction	Résultat attendu
`moyenne`	`0.0`
`chercherMaximum`	`false`
`compterHorsPlage`	`0`

Pour aller plus loin : ajoutez au module une fonction `void afficherMesures(const std::vector<Mesure>& mesures)`, puis justifiez pourquoi elle ne doit **pas** être utilisée dans les tests automatiques.